ONLINE BOUTIQUE ON AMAZON EKS

Operations Runbook

How to run, check, change and stop the platform

Document ID	OBE-RUN-001
Version	2.1
Date	2026-10-10
Status	Approved for the dev environment
Owner	khaipd18 (DevOps / Cloud)
Audience	Engineers who deploy or operate the platform
Classification	Internal
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Document control

Revision history

Version	Date	Change	Author
1.0	2026-10-09	First issue.	khaipd18
1.1	2026-10-10	First run on the new account; managed control plane log group.	khaipd18
1.2	2026-10-10	Screenshots from the full EKS deployment; IR-15.	khaipd18
2.0	2026-10-10	Rewritten to be shorter: the main tasks in plain steps, all problems in one table, advanced topics moved to the appendix.	khaipd18
2.1	2026-10-10	CI writes image tags through pull requests; no need to switch off the branch rule any more.	khaipd18

Related documents

- OBE-TDD-001 Technical Design Document: why the platform is built this way
- README.md: project summary and how to test locally with kind

Contents

1 Start here 3
1.1 What runs where 3
1.2 Three rules 3
1.3 Tools you need 3
2 Main tasks 4
2.1 Start the environment 4
2.2 Check that everything is healthy 4
2.3 Release a new version of a service 7
2.4 Roll back a service 7
2.5 Change settings or scale a service 7
2.6 Change the infrastructure 8
2.7 Stop the environment 8
3 When something goes wrong 10
Appendix 12
A. Upgrade Kubernetes 12
B. Security findings 12
C. Cost 13
D. Recovery 13
E. Services and ports 13
F. Versions 14

1 Start here

1.1 What runs where

Part	Where it lives
Infrastructure (VPC, EKS, ECR, IAM)	AWS `ap-southeast-1`, created by Terraform from `terraform/`
Kubernetes cluster	`khaipd18-eks-cluster`, 3 `t3.medium` nodes
Shop (12 services)	Namespace `dev-eks`, deployed by Argo CD from `gitops/dev-eks/`
Argo CD	Namespace `argocd`, inside the cluster
Monitoring (Prometheus, Grafana)	Namespace `monitoring`
CI/CD	GitHub Actions in the repository

1.2 Three rules

1. **Change things through Git.** Argo CD puts the cluster back to what Git says, so a manual `kubectl` change is undone within minutes.
2. **Infrastructure changes go through a pull request.** The Terraform apply waits for your approval in GitHub.
3. **Stop the environment when you are done.** It costs about 0.40 USD per hour plus the load balancer (Appendix C).

1.3 Tools you need

AWS CLI v2, Terraform 1.14+, `kubectl`, Helm, GitHub CLI (`gh`). Log in to AWS with the account's profile and check it:

```
aws sts get-caller-identity          # shows the account you are using
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl get nodes                    # 3 nodes, all Ready
```

> **NOTE**
> Only the identity that created the cluster is cluster admin. The role used in the AWS console gets read-only access automatically (Terraform), so the EKS console can show pods and nodes.

2 Main tasks

2.1 Start the environment

Time: about 45 minutes. **Cost starts at step 3.**

1. **Tell GitHub which AWS account to use:**

```
gh variable set AWS_ACCOUNT_ID --body <account-id>
```

2. **Create the place where Terraform keeps its state** (only once per account):

```
aws s3api create-bucket --bucket <state-bucket> --region ap-southeast-1 \
  --create-bucket-configuration LocationConstraint=ap-southeast-1
aws s3api put-bucket-versioning --bucket <state-bucket> \
  --versioning-configuration Status=Enabled
aws dynamodb create-table --table-name <lock-table> --region ap-southeast-1 \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH --billing-mode PAY_PER_REQUEST
```

   The names must match `terraform/backend.tf`.

3. **Create the infrastructure** (about 17 minutes):

```
cd terraform
terraform init
terraform plan -out tfplan     # read it: 74 resources to add
terraform apply tfplan
```

4. **Install Argo CD and tell it what to deploy:**

```
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl create namespace argocd
kubectl apply -n argocd --server-side -f https://raw.githubusercontent.com/argoproj/
argo-cd/stable/manifests/install.yaml
kubectl apply -f gitops/argocd/namespaces.yaml
kubectl apply -f gitops/argocd/applicationset.yaml
kubectl apply -f gitops/argocd/monitoring.yaml --server-side
```

5. **Build the images.** CI pushes them to ECR and writes the new tags into Git through small pull requests that it merges itself:

```
for w in go dotnet java node py; do gh workflow run $w-services-ci.yaml --ref main;
done
gh run list --limit 5            # wait until the 5 runs are green
```

6. **Check it works:** follow the next section. Everything should look like the pictures there.

2.2 Check that everything is healthy

Run these four commands:

```
kubectl get nodes                       # 3 nodes, all Ready
kubectl -n argocd get applications      # 14 apps, all Synced and Healthy
kubectl -n dev-eks get pods             # all Running, restarts not growing
kubectl -n dev-eks get hpa              # frontend CPU below 70%
```

```
$ kubectl get nodes
NAME                                          STATUS   ROLES    AGE   VERSION
ip-10-18-2-103.ap-southeast-1.compute.internal   Ready    <none>   19m   v1.35.8-eks-3b4a6ca
ip-10-18-2-151.ap-southeast-1.compute.internal   Ready    <none>   19m   v1.35.8-eks-3b4a6ca
ip-10-18-3-199.ap-southeast-1.compute.internal   Ready    <none>   19m   v1.35.8-eks-3b4a6ca

$ kubectl -n dev-eks get pods
NAME                                     READY   STATUS    RESTARTS   AGE
adservice-dev-6f964d9445-7h6fp           1/1     Running   0          6m15s
cartservice-dev-659646cb65-wmx8c         1/1     Running   0          6m15s
checkoutservice-dev-856956f79f-qqm5v     1/1     Running   0          6m50s
currencyservice-dev-6fdb5dbc49-g26dr     1/1     Running   0          6m46s
emailservice-dev-799c485bc8-kjsnp        1/1     Running   0          6m55s
frontend-dev-59d4f86fc6-7hrfh            1/1     Running   0          6m11s
frontend-dev-59d4f86fc6-9ddkc            1/1     Running   0          5m46s
paymentservice-dev-75479bc798-w6rvc      1/1     Running   0          6m4s
productcatalogservice-dev-64c9b8d476-c2pfz   1/1     Running   0          6m4s
recommendationservice-dev-67c9ffc4c8-6zhs5   1/1     Running   0          6m46s
redis-cart-dev-65df6bd654-pxv74          1/1     Running   0          10m
shippingservice-dev-78b748699b-d2dfb     1/1     Running   0          6m47s

$ kubectl -n dev-eks get hpa,pdb
NAME                                                REFERENCE                  TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/frontend-dev    Deployment/frontend-dev    cpu: 1%/70%   2         4         2          10m

NAME                                    MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/frontend-dev   1               N/A               1                     10m

$ kubectl top nodes
NAME                                          CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
ip-10-18-2-103.ap-southeast-1.compute.internal   55m          2%       1278Mi          38%
ip-10-18-2-151.ap-southeast-1.compute.internal   78m          4%       1630Mi          49%
ip-10-18-3-199.ap-southeast-1.compute.internal   80m          4%       1440Mi          43%
```

Figure 1: What a healthy cluster looks like

Open the shop: get its address with `kubectl -n dev-eks get svc frontend-external-dev`, open the `EXTERNAL-IP` hostname in a browser and place an order.

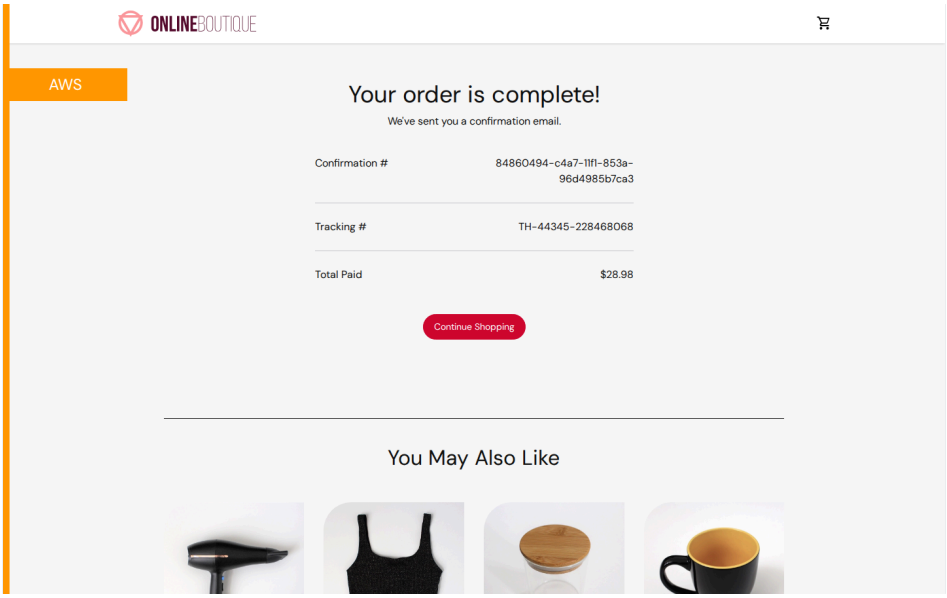


Figure 2: A test order went through: the whole shop works

Open the web consoles (each command keeps running; open the address in your browser):

Console	Command	Address
Argo CD	`kubectl -n argocd port-forward svc/argocd-server 8080:443`	https://localhost:8080
Grafana	`kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80`	http://localhost:3000

User `admin`. Passwords:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}'
| base64 -d; echo
kubectl -n monitoring get secret kube-prometheus-stack-grafana -o
jsonpath='{.data.admin-password}' | base64 -d; echo
```

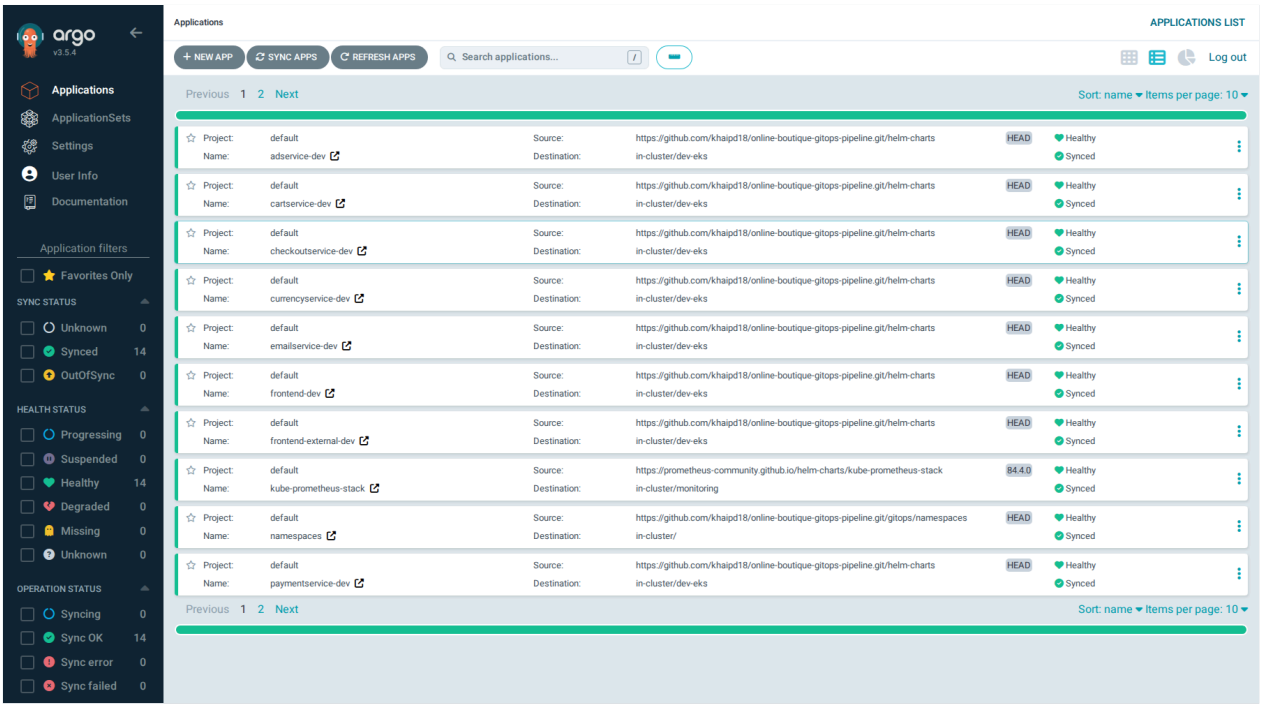


Figure 3: Argo CD: every application Synced and Healthy

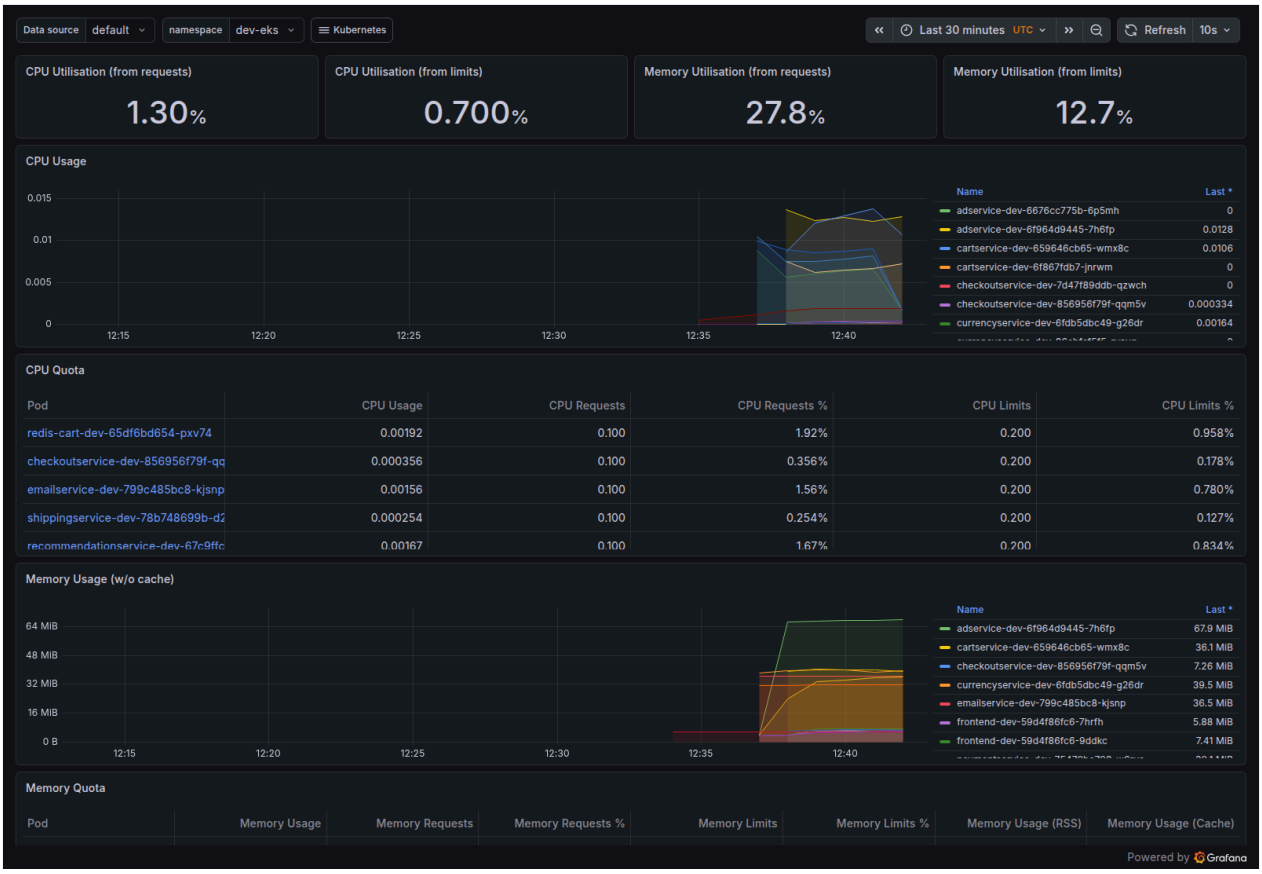


Figure 4: Grafana, dashboard “Kubernetes / Compute Resources / Namespace (Pods)”: CPU and memory per pod

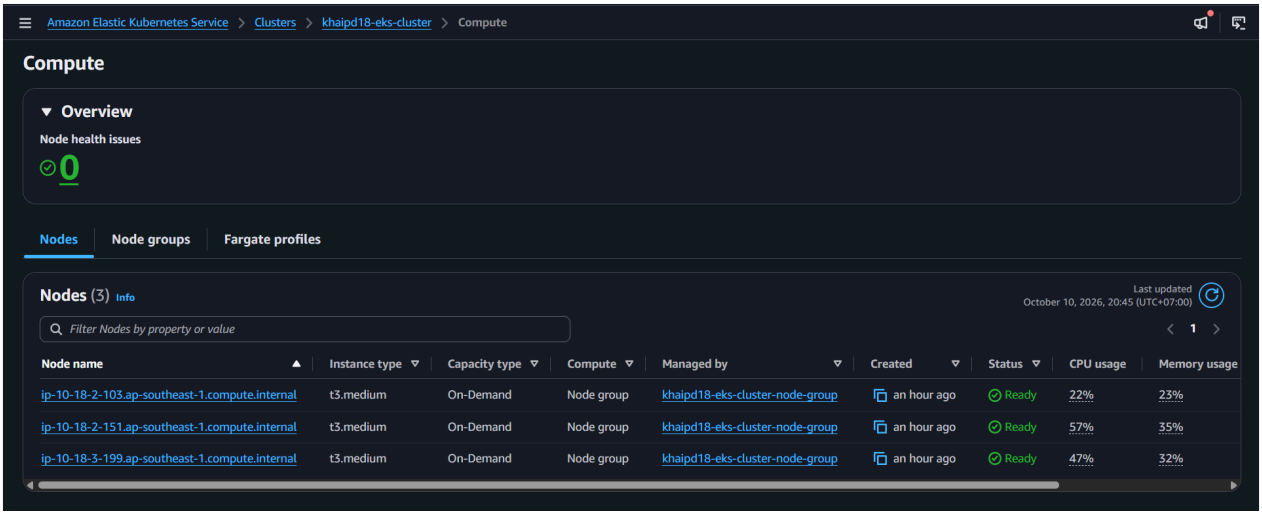


Figure 5: AWS console, EKS → Compute: three nodes Ready

2.3 Release a new version of a service

1. Merge your change under `src/<service>/` into `main`.
2. GitHub Actions tests, builds and scans the image, pushes it to ECR and writes the new tag into `gitops/dev-eks/values-<service>.yaml` through a pull request it merges itself.
3. Argo CD sees the new tag within about 3 minutes and updates the pods.

Check: `kubectl -n dev-eks rollout status deploy/<service>-dev`

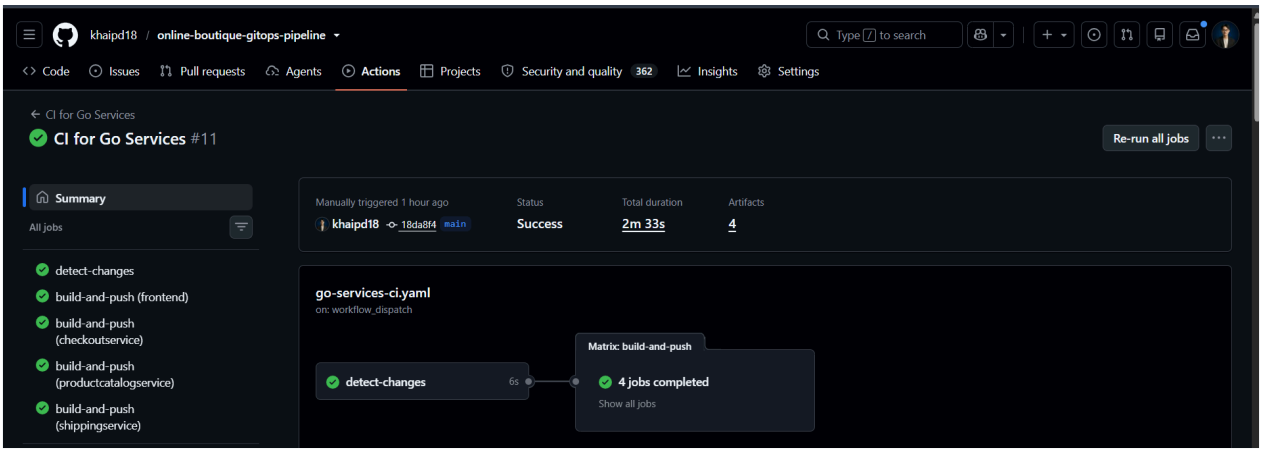


Figure 6: A successful CI run in GitHub Actions

2.4 Roll back a service

Every image tag is a commit, so going back means restoring the previous tag in Git:

```
git log --oneline -- gitops/dev-eks/values-<service>.yaml   # find the bad tag commit
git revert <commit>                                          # then push or open a PR
```

> **WARNING**
> Do not use `kubectl set image` or `kubectl rollout undo`. Argo CD will put the bad version back.

2.5 Change settings or scale a service

All settings of a service are in `gitops/dev-eks/values-<service>.yaml`: environment variables, CPU and memory, number of replicas, autoscaling, and which services may call it (`networkPolicy.allowFrom`).

1. Edit the file in a branch.
2. Check the result: `helm template <service>-dev helm-charts -f gitops/dev-eks/values-<service>.yaml`
3. Open a pull request and merge it. Argo CD applies it.

> **TIP**
> If a service starts calling another one, add the caller to the other service's `allowFrom`, otherwise the call is blocked.

To change the number of **nodes**, edit `eks_node_group_scaling_config` in `terraform/variables.tf` and follow the next section.

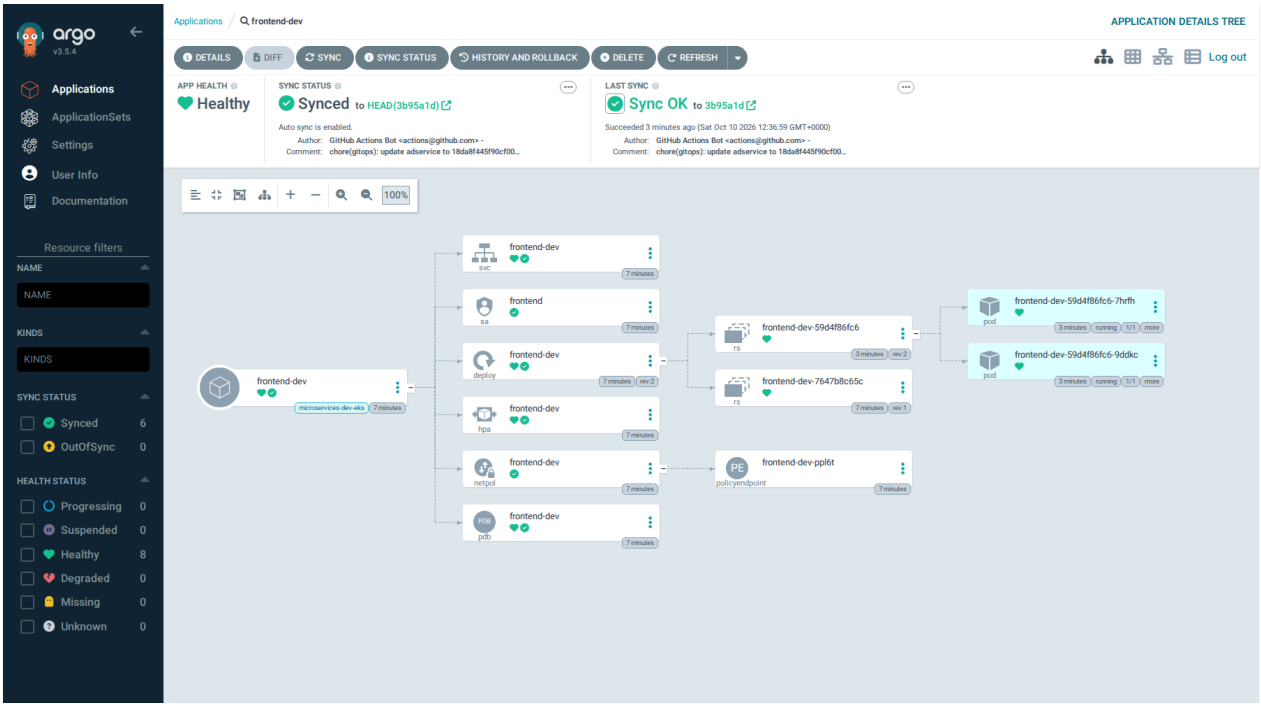


Figure 7: Argo CD shows everything one service owns: Deployment, pods, HPA, NetworkPolicy, PodDisruptionBudget

2.6 Change the infrastructure

1. Edit `terraform/` in a branch and open a pull request.
2. GitHub runs Checkov (security check) and `terraform plan`. Read the plan in the job log.
3. Merge the pull request.
4. The **Terraform Apply** job waits. In GitHub: **Actions** → the run → **Review deployments** → **production** → **Approve**.

> **IMPORTANT**
> Only approve a plan you have read. The apply runs with full admin rights.

2.7 Stop the environment

1. Remove the applications and the load balancer first (Terraform cannot delete the VPC while the load balancer exists):

```
kubectl delete -f gitops/argocd/applicationset.yaml
kubectl -n dev-eks delete svc frontend-external-dev --ignore-not-found
```

2. Delete the infrastructure (about 10 minutes):

```
cd terraform && terraform destroy
```

3. Tell CI there is no AWS account any more: `gh variable delete AWS_ACCOUNT_ID`

Check: `aws eks list-clusters --region ap-southeast-1` returns an empty list. The state bucket and lock table stay; they cost almost nothing.

3 When something goes wrong

Find the symptom in the table, then apply the fix. The command in the middle column usually shows the reason.

Symptom	Likely reason / how to see it	Fix
CI fails at "Configure AWS credentials"	Wrong `AWS_ACCOUNT_ID`, or the IAM roles do not exist yet. `gh variable list`	Fix the variable, or create the roles with the first `terraform apply`
CI fails at "Update GitOps through a pull request"	GitHub Actions may not create pull requests in this repository	Settings → Actions → General → tick "Allow GitHub Actions to create and approve pull requests"
Terraform: `Error acquiring the state lock`	An earlier run stopped half-way. Make sure nothing else runs: `gh run list`	`terraform force-unlock <lock-id>`
Terraform Apply job is **Waiting**	It needs your approval	Approve it in GitHub (Change the infrastructure)
Argo CD app **OutOfSync** or **Degraded**	Bad values or a pod that does not start. `kubectl -n argocd describe application <app>`	Fix the values in Git
Pod in `ImagePullBackOff`	The image tag is not in ECR. `kubectl -n dev-eks describe pod <pod>`	Run the service's CI, or roll back
Pod in `CrashLoopBackOff` / `OOMKilled`	Wrong setting or not enough memory. `kubectl -n dev-eks logs <pod> --previous`	Fix the values or roll back
Pod `forbidden`: `violates PodSecurity`	The values remove the safe security settings	Remove that override from the values file
Calls between services time out	The caller is not in `allowFrom`. `kubectl -n dev-eks get networkpolicy`	Add the caller to `allowFrom`
Shop address does not open	Load balancer still starting, or `frontend` is down. `kubectl -n dev-eks get pods`	Wait a few minutes; fix `frontend`
Pods stay `Pending`	Not enough room on the nodes. `kubectl describe pod <pod>`	Reduce replicas or add a node
HPA shows `<unknown>`	Metrics Server not ready. `kubectl top pods -n dev-eks`	Re-apply Terraform (it installs Metrics Server)
`kubectl drain` never finishes	The PodDisruptionBudget keeps one `frontend` pod alive (on purpose)	Add a node so the pod can move, then retry
Shopping carts are empty	`redis-cart` restarted; carts are not saved to disk	Expected in this demo
EKS console shows "Data unavailable"	The console role has no access in the cluster	Re-apply Terraform (it gives the console role read access)

```
$ kubectl -n dev-eks run psa-test --image=busybox:1.37 --restart=Never -- sleep 60
Error from server (Forbidden): pods "psa-test" is forbidden: violates PodSecurity "restricted:latest": allowPrivilegeEscalation !=
false (container "psa-test" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "psa-
test" must set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container "psa-test" must set
securityContext.runAsNonRoot=true), seccompProfile (pod or container "psa-test" must set securityContext.seccompProfile.type to
"RuntimeDefault" or "Localhost")

$ kubectl -n dev-eks exec np-test -- nc -zv -w 3 frontend-dev 80
frontend-dev (172.20.100.89:80) open

$ kubectl -n dev-eks exec np-test -- nc -zv -w 3 paymentservice-dev 50051
nc: paymentservice-dev (172.20.151.236:50051): Connection timed out
command terminated with exit code 1

$ kubectl -n dev-eks exec np-test -- nc -zv -w 3 redis-cart-dev 6379
nc: redis-cart-dev (172.20.141.193:6379): Connection timed out
command terminated with exit code 1
```

Figure 8: Security working as designed: a pod without security settings is rejected; a pod that is not allowed cannot reach `paymentservice` or `redis-cart`

IMPORTANT

If users are affected (shop down, checkout broken), roll back first and investigate after. Write down what happened in a GitHub issue.

Appendix

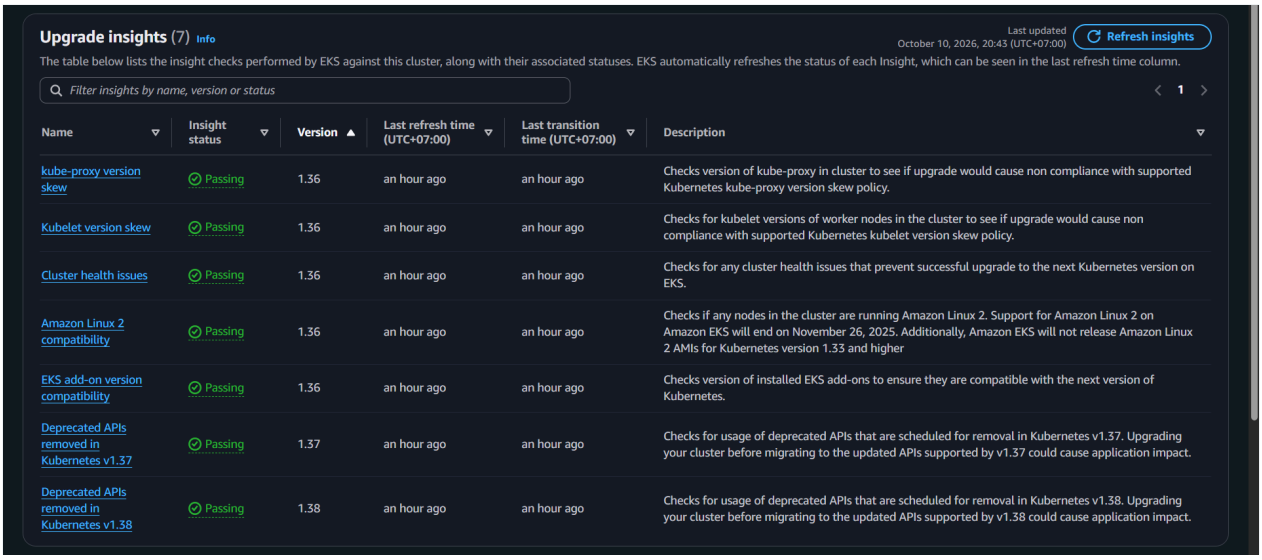
A. Upgrade Kubernetes

Kubernetes 1.35 is supported by EKS until 27 March 2027 [AWS]. Upgrade one version at a time:

1. Check **EKS console → Upgrade insights**: every check must be **Passing**.
2. Change `eks_k8s_version` in Terraform and apply it. This upgrades the control plane.
3. Upgrade the nodes; Terraform does not do it [AWS]:

```
aws eks update-nodegroup-version --cluster-name khaipd18-eks-cluster \
  --nodegroup-name khaipd18-eks-cluster-node-group --kubernetes-version <version>
```

4. Update the add-on versions (`eks_*_version` variables). List the versions with `aws eks describe-addon-versions --addon-name <name> --kubernetes-version <version>`.



Upgrade insights (7) Info

Last updated October 10, 2026, 20:43 (UTC+07:00) — Refresh insights

The table below lists the insight checks performed by EKS against this cluster, along with their associated statuses. EKS automatically refreshes the status of each Insight, which can be seen in the last refresh time column.

Filter insights by name, version or status — 1

Name	Insight status	Version	Last refresh time (UTC+07:00)	Last transition time (UTC+07:00)	Description
kube-proxy version skew	Passing	1.36	an hour ago	an hour ago	Checks version of kube-proxy in cluster to see if upgrade would cause non compliance with supported Kubernetes kube-proxy version skew policy.
Kubelet version skew	Passing	1.36	an hour ago	an hour ago	Checks for kubelet versions of worker nodes in the cluster to see if upgrade would cause non compliance with supported Kubernetes kubelet version skew policy.
Cluster health issues	Passing	1.36	an hour ago	an hour ago	Checks for any cluster health issues that prevent successful upgrade to the next Kubernetes version on EKS.
Amazon Linux 2 compatibility	Passing	1.36	an hour ago	an hour ago	Checks if any nodes in the cluster are running Amazon Linux 2. Support for Amazon Linux 2 on Amazon EKS will end on November 26, 2025. Additionally, Amazon EKS will not release Amazon Linux 2 AMIs for Kubernetes version 1.33 and higher
EKS add-on version compatibility	Passing	1.36	an hour ago	an hour ago	Checks version of installed EKS add-ons to ensure they are compatible with the next version of Kubernetes.
Deprecated APIs removed in Kubernetes v1.37	Passing	1.37	an hour ago	an hour ago	Checks for usage of deprecated APIs that are scheduled for removal in Kubernetes v1.37. Upgrading your cluster before migrating to the updated APIs supported by v1.37 could cause application impact.
Deprecated APIs removed in Kubernetes v1.38	Passing	1.38	an hour ago	an hour ago	Checks for usage of deprecated APIs that are scheduled for removal in Kubernetes v1.38. Upgrading your cluster before migrating to the updated APIs supported by v1.38 could cause application impact.

Figure 9: EKS upgrade insights: all checks passing

B. Security findings

- **Checkov (Terraform):** fix the code if you can. If a finding is acceptable, write down why and update the baseline in a pull request: `checkov --config-file .checkov.yaml -d terraform --framework terraform --create-baseline`
- **Trivy (images):** findings are in GitHub → **Security** → **Code scanning**. They come from the application code in `src/`, which this repository does not change, so they are reported but do not block CI.

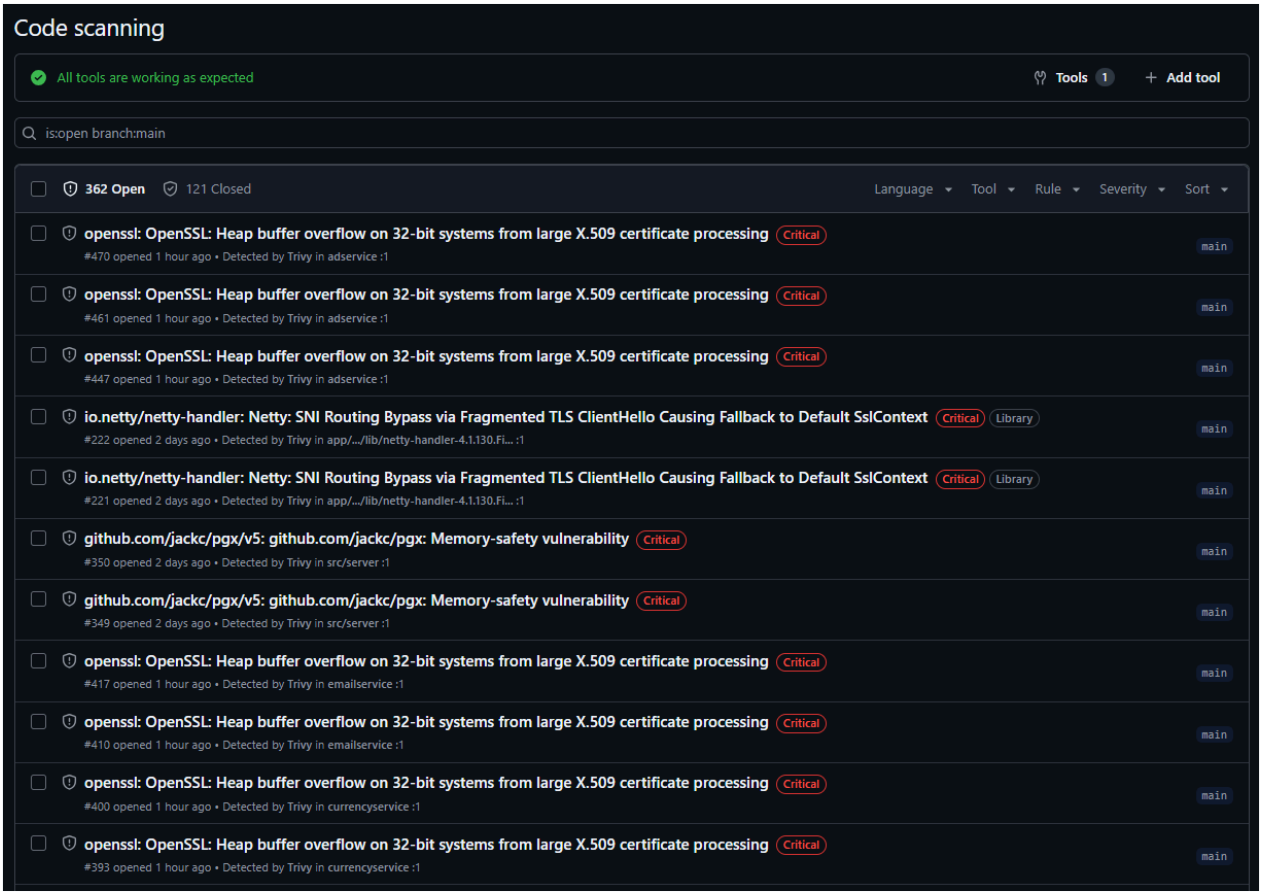


Figure 10: GitHub code scanning: Trivy findings per image

C. Cost

About **0.40 USD per hour** while the environment runs, before the load balancer and small items (on-demand prices in `ap-southeast-1` from the AWS Price List API): EKS control plane 0.10, three `t3.medium` nodes 0.16, six interface endpoint ENIs 0.08, NAT Gateway 0.06. See the real spend in AWS Cost Explorer. Stop the environment when it is not needed.

D. Recovery

Lost	How to get it back
The whole cluster or account	Start the environment again: everything is rebuilt from Git in about 45 minutes
Terraform state	The state bucket keeps old versions: restore the previous version of `dev/terraform.tfstate`
A bad deployment	Roll back the service
Shopping carts, Grafana history	Not saved; they are lost

E. Services and ports

Service	Language	Port	Accepts calls from	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi

productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi
shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

F. Versions

Component	Version	Defined in
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml